An early look at the LDBC Social Network Benchmark's Business Intelligence workload

Gábor Szárnyas, Arnau Prat-Pérez, Alex Averbuch, József Marton, Marcus Paradies, Moritz Kaufmann, Orri Erling, Peter Boncz, Vlad Haprian, János Benjamin Antal

GRADES-NDA @ SIGMOD
Houston, TX

Hungarian Academy of Sciences

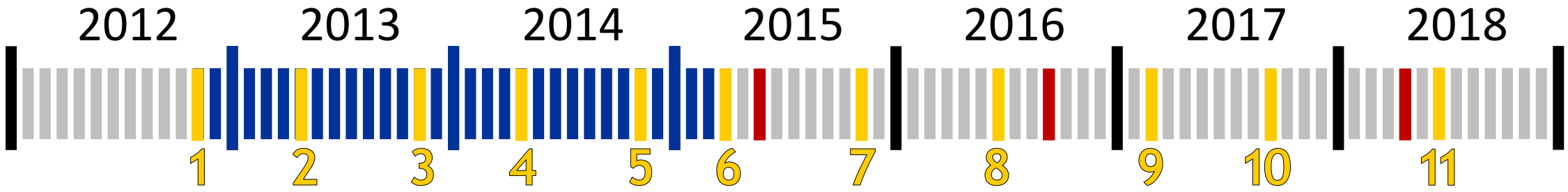
McGill

Linked Data Benchmark Council

LDBC is a non-profit organization dedicated to establishing benchmarks, benchmark practices and benchmark results for graph data management SW.

LDBC's Social Network Benchmark is an industrial and academic initiative, formed by principal actors in the field of graph-like data management.

LDBC timeline



EU FP7 project

TUC meetings

Benchmark papers

30+ papers in total, including G-CORE (presented on Thursday)

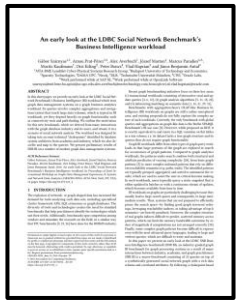
Interactive
SIGMOD 2015


The LDBC Social Network Benchmark: Interactive Workload

Graphalytics
VLDB 2016


LDBC Graphalytics: A Benchmark for Large-Scale Graph Analysis on Parallel and Distributed Platforms

Business Intelligence
GRADES-NDA @SIGMOD 2018


An early look at the LDBC Social Network Benchmark's Business Intelligence workload

Graph processing landscape

LDBC

Interactive	local queries
BI	global queries
Graphalytics	computations

Graph processing landscape

LDBC

Interactive	local queries	limited data	frequent upd.

Example: “Recently liked by friends”

```
MATCH
 (u:User {id: $uID})-[:FRIEND]-(f:User)-[l:LIKES]->(p:Post)
RETURN f, p
ORDER BY l.timestamp DESC
LIMIT 10
```

BI	global queries
Graphalytics	computations

Graph processing landscape

LDBC

Interactive	local queries	limited data	frequent upd.
BI	global queries	lots of data	infreq. upd.

Example: “One-sided friendships”

```
MATCH (u1:User)-[:FRIEND]-(u2:User)-[l:LIKES]->(p:Post),
      (u1)-[:AUTHOR_OF]->(p)
WITH u1, u2, count(l) AS likes
WHERE likes > 10
  AND NOT (u1)-[:LIKES]->(:Post)<-[:AUTHOR_OF]-(u2)
RETURN u1, u2
```

Graphalytics	computations

Graph processing landscape

LDBC

Interactive	local queries	limited data	frequent upd.
BI	global queries	lots of data	infreq. upd.
Graphalytics	computations	all data	no upd.

Example: “Find the most central individuals.”

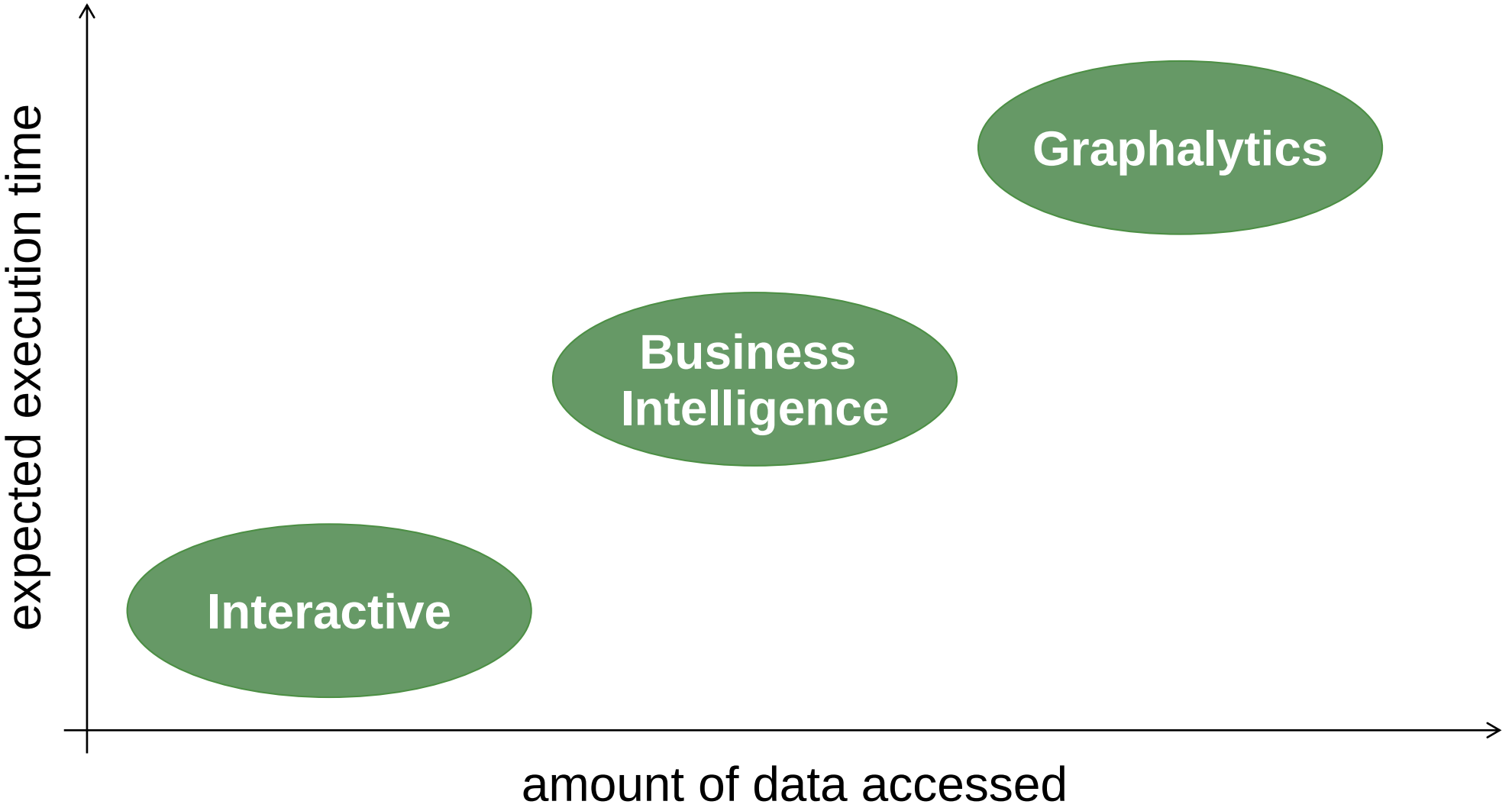
BFS breadth-first search
PR PageRank
CDLP community detection by label propagation
WCC weakly connected components
LCC local clustering coefficient
SSSP single-source shortest path

Graph processing landscape

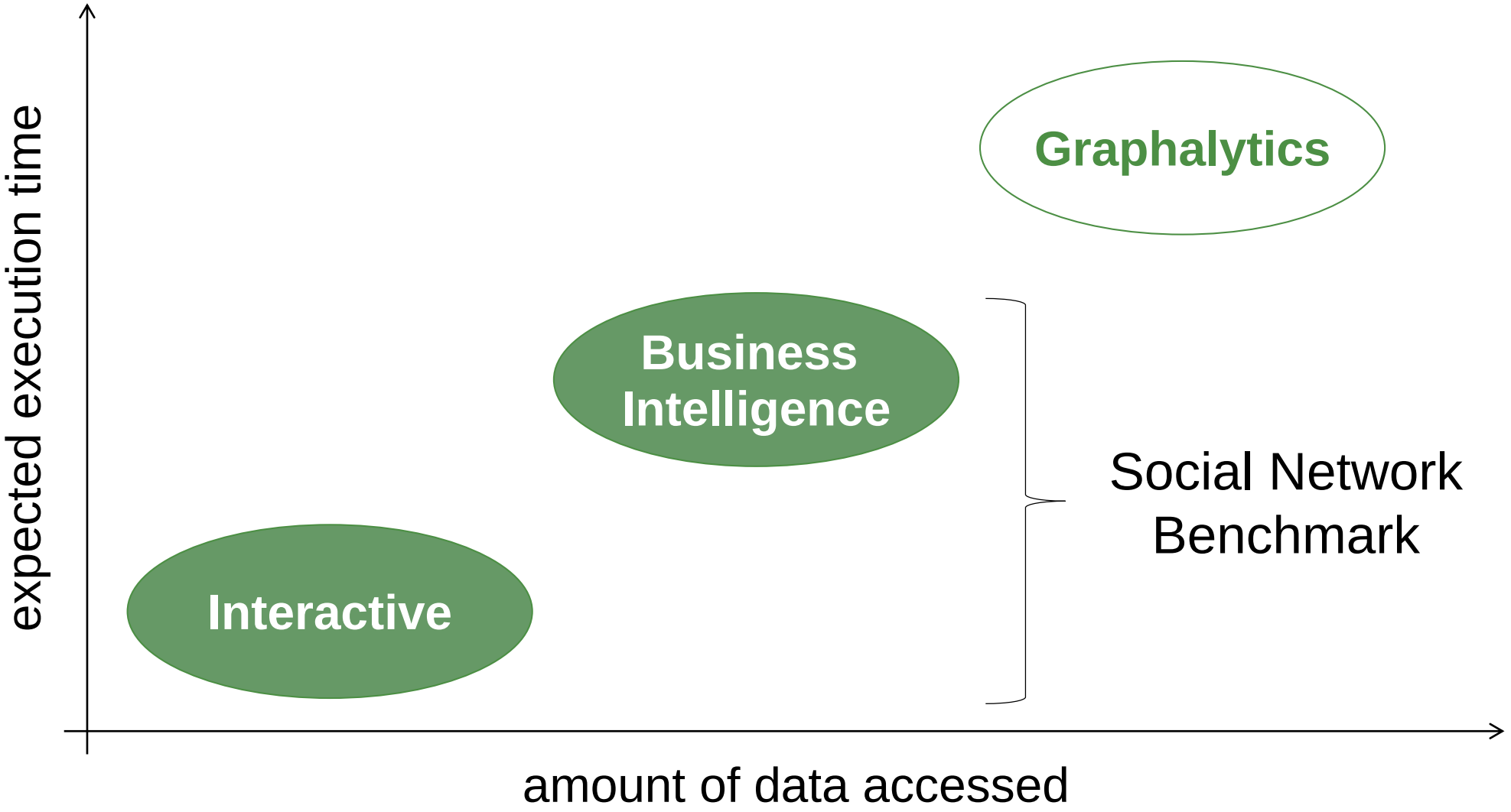
LDBC

Interactive	local queries	limited data	lots of upd.
BI	global queries	lots of data	few upd.
Graphalytics	computations	all data	no upd.

LDBC benchmarks at a glance



LDBC benchmarks at a glance



Social network graph

LDBC

DATAGEN:

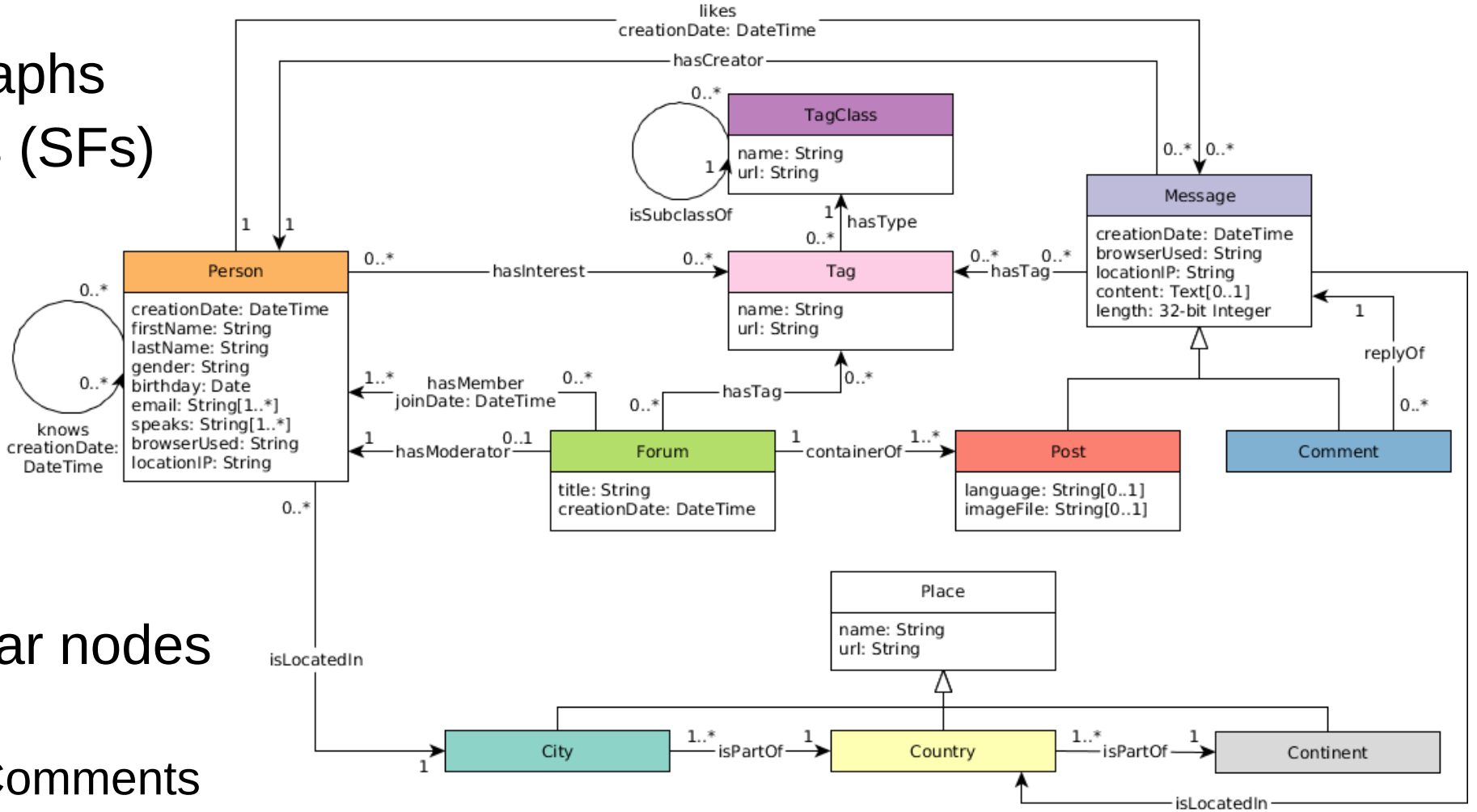
- Generate realistic graphs
- Multiple scale factors (SFs)

Nodes:

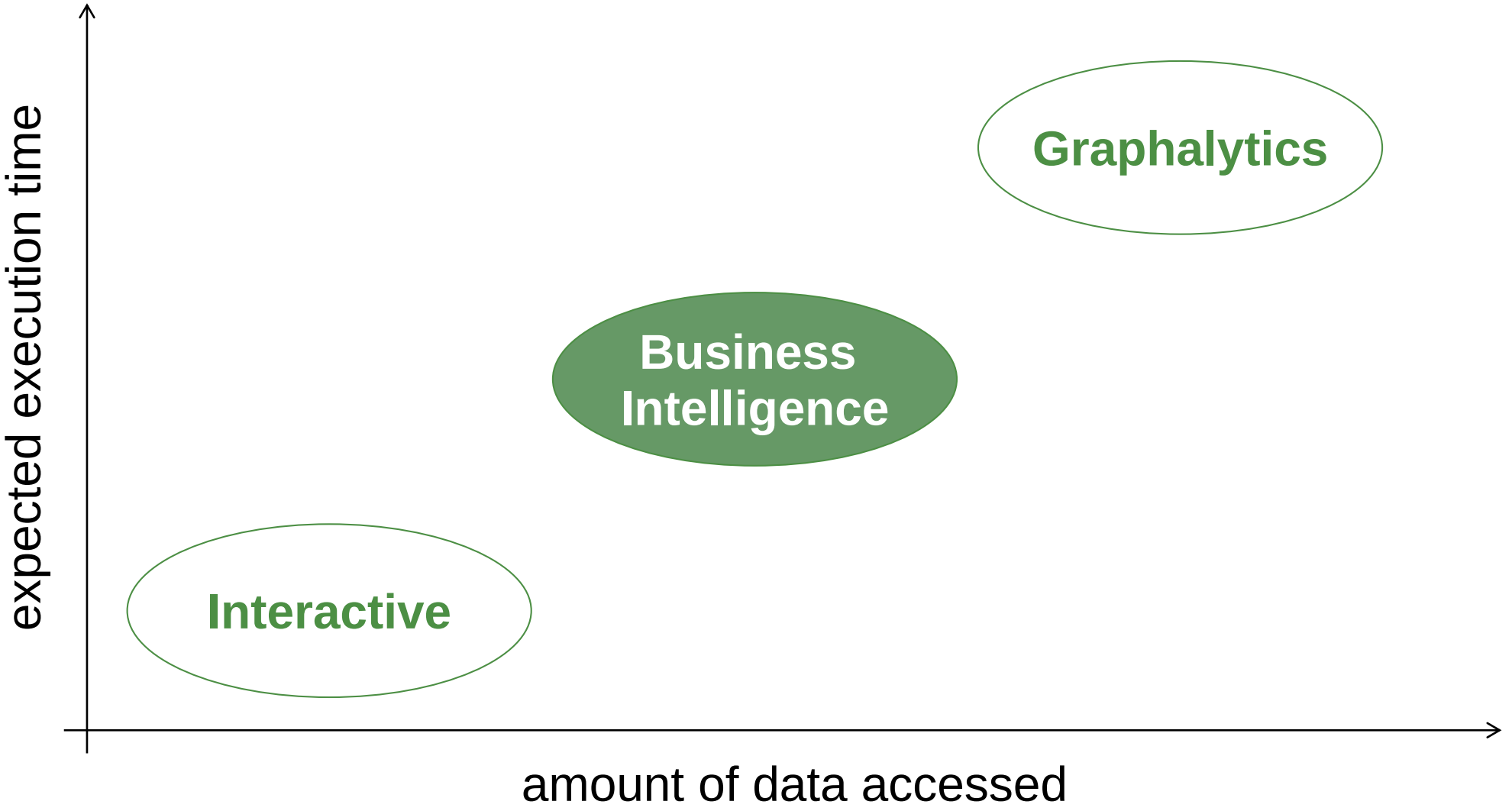
- Collection attributes
- Type inheritance

Edges:

- With attributes
- Edges between similar nodes
  - Network of Persons
  - Reply tree of Posts/Comments

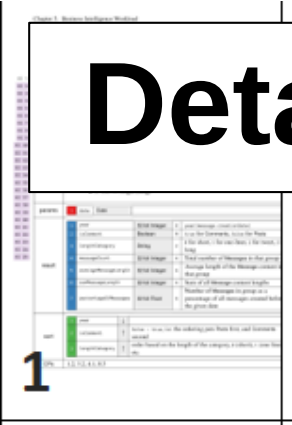


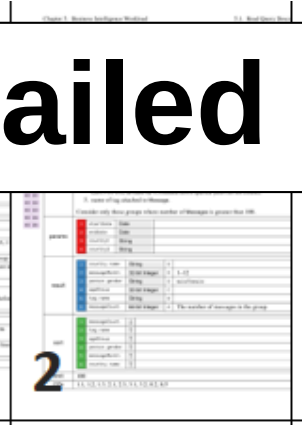
LDBC benchmarks at a glance

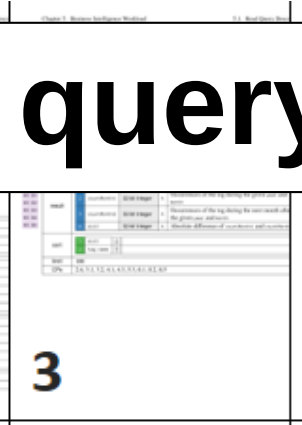


Detailed query specifications

LDBC

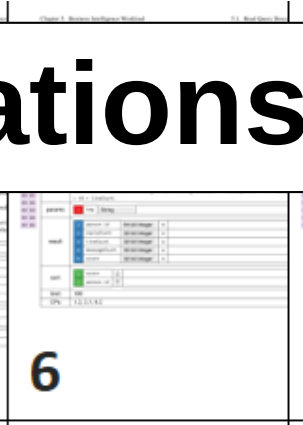

1

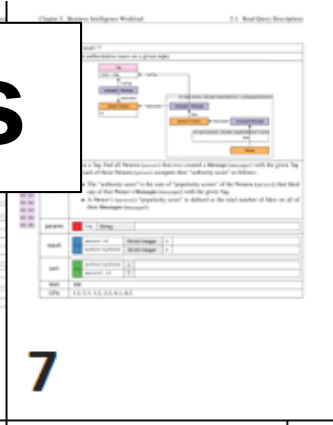

2

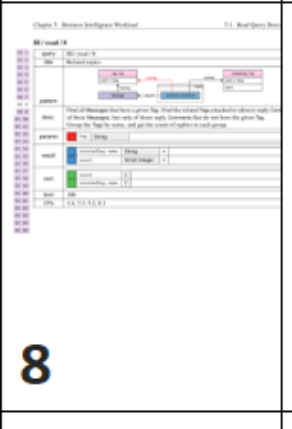

3

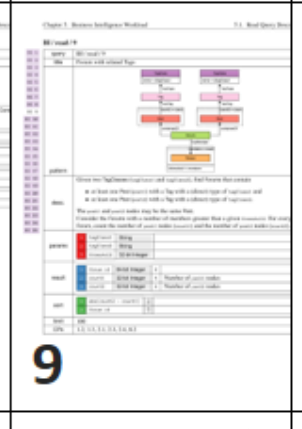

4

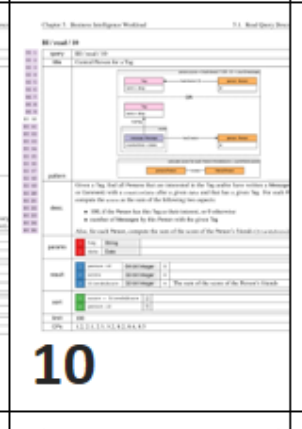

5

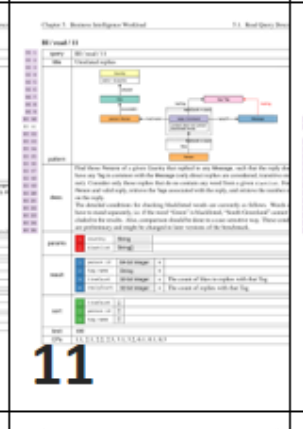

6

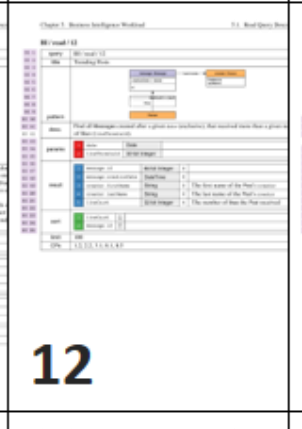

7

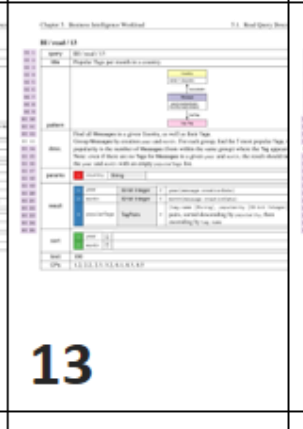

8

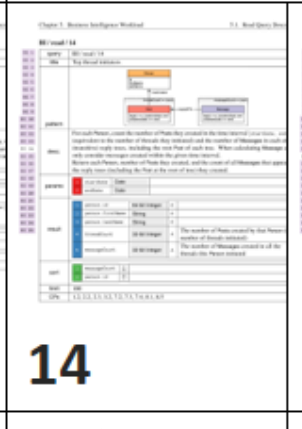

9

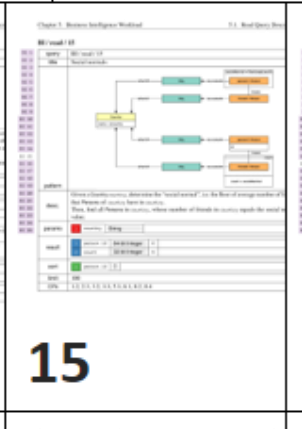

10

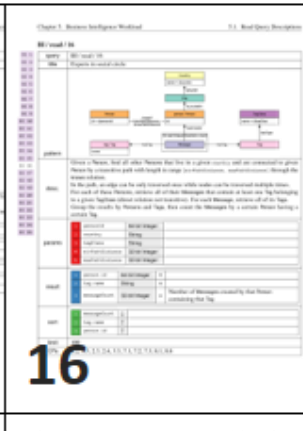

11

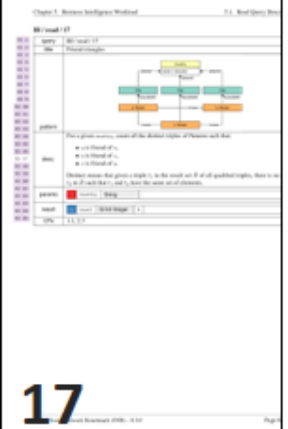

12

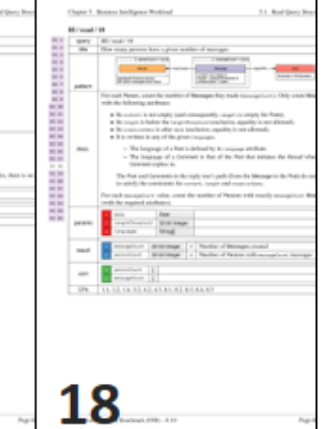

13

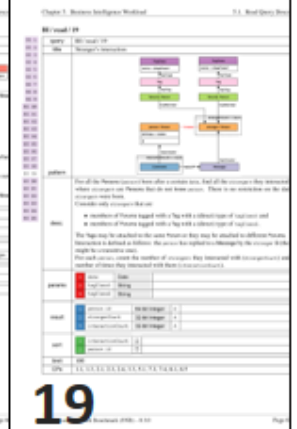

14


15

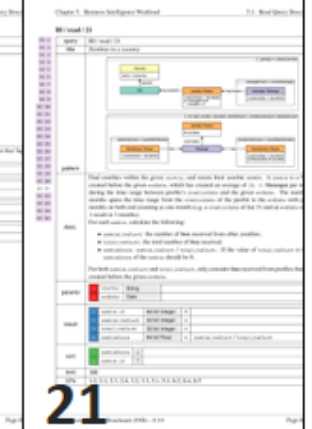

16

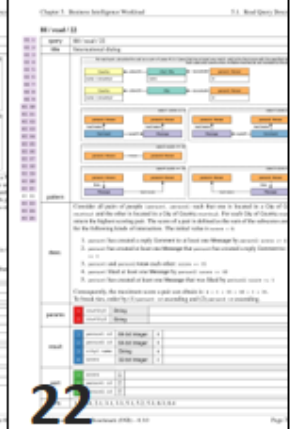

17

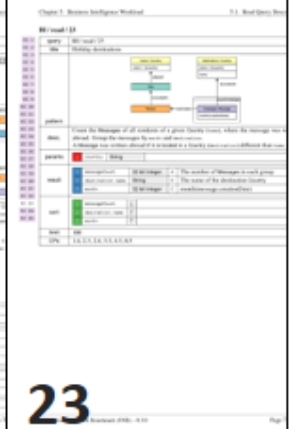

18

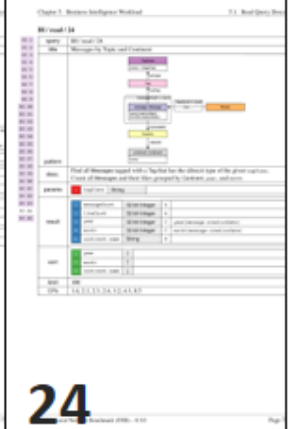

19

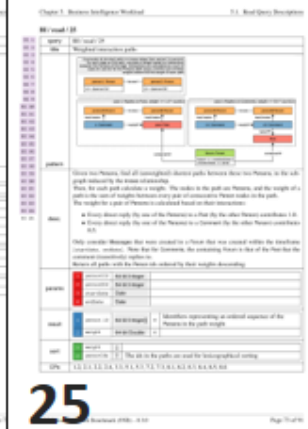

20

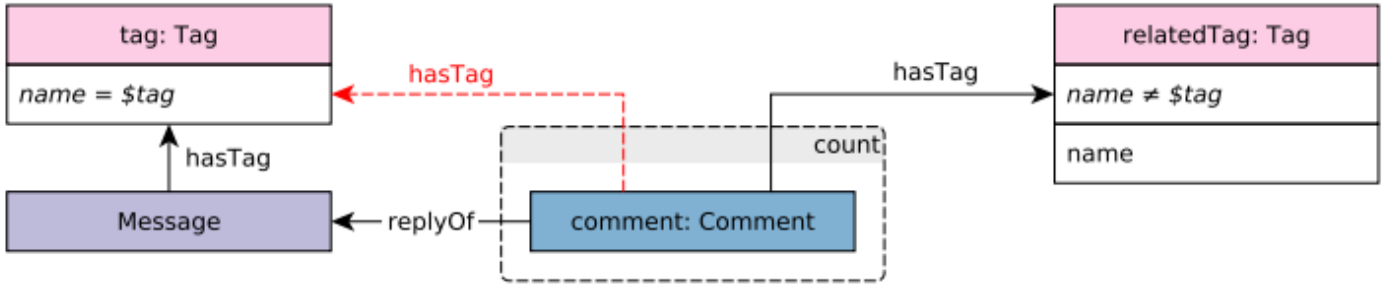

21


22


23


24


25

query	BI / read / 8
title	Related topics
pattern	
desc.	Find all Messages that have a given Tag. Find the related Tags attached to (direct) reply Comments of these Messages, but only of those reply Comments that do not have the given Tag. Group the Tags by name, and get the count of replies in each group.

params

1	tag	String	

result

1	relatedTag.name	String	R	
2	count	32-bit Integer	A	

sort

1	count	↓	
2	relatedTag.name	↑	

limit	100
CPs	1.4, 3.3, 5.2, 8.1

design starts here

Choke points

- = a challenging aspect of query processing, a well-chosen difficulty
- Allows systematic benchmark design

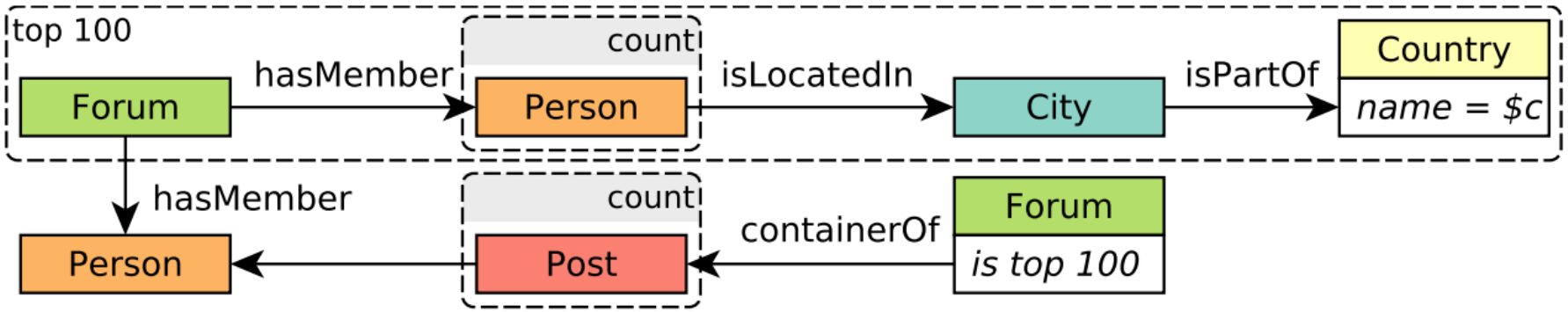
CP-2.1: [QOPT] Rich join order optimization

TPC-H 2.3

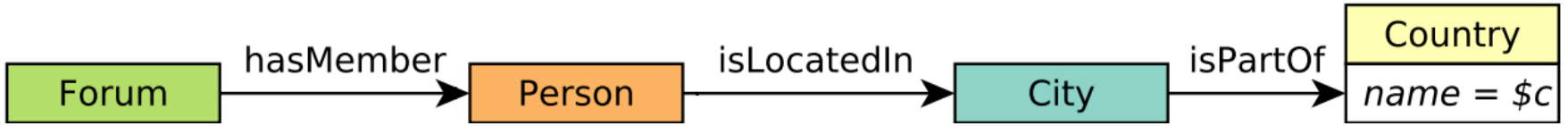
This choke-point tests the ability of the query optimizer to find optimal join orders. A graph can be traversed in different ways. In the relational model, this is equivalent as different join orders. The execution time of these orders may differ by orders of magnitude. Therefore, finding an efficient join (traversal) order is important, which in general, requires enumeration of all the possibilities. The enumeration is complicated by operators that are not freely re-orderable like semi-, anti-, and outer-joins. Because of this difficulty most join enumeration algorithms do not enumerate all possible plans, and therefore can miss the optimal join order. Therefore, these chokepoint tests the ability of the query optimizer to find optimal join (traversal) orders.

Peter Boncz, Thomas Neumann, Orri Erling,
TPC-H Analyzed: Hidden Messages and Lessons Learned from an Influential Benchmark,
TPCTC 2013

Q5: Top posters in a country



1. Find the *top 100* Forums by members in a given Country.
2. For each member of the *top 100* Forums, count their Posts in the *top 100* Forums.



Sparksee SF10

Forum to Country: 1 → 2 → 3 → 4 — 57 s

Country to Forum: 4 → 3 → 2 → 1 — 0.3 s

CP-2.1 Rich join order optimization

Choke points: optimizations

- “Top-*k* pushdown” optimization
- New in LDBC (not covered in TPC-H choke points)

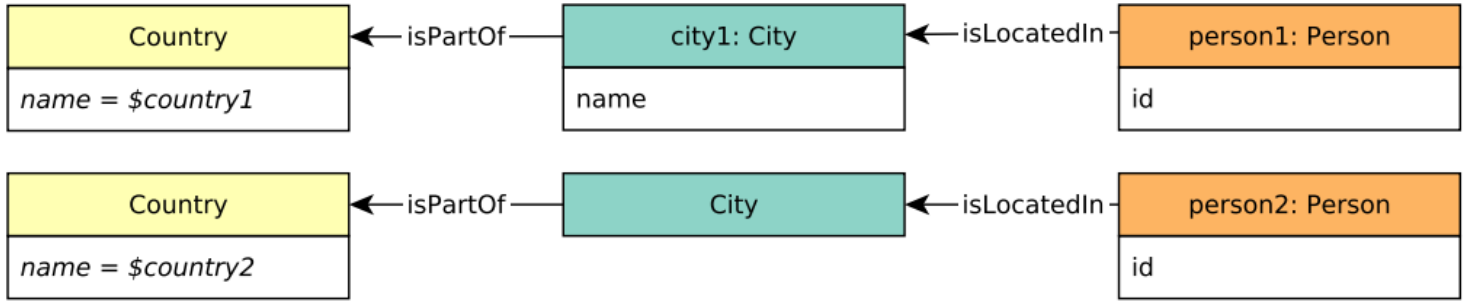
CP-1.3: [QOPT] Top-k pushdown

This choke-point tests the ability of the query optimizer to perform optimizations based on top-k selections. Many times queries demand for returning the top-k elements based on some property. Engines can exploit that once k results are obtained, extra restrictions in a selection can be added based on the properties of the kth element currently in the top-k, being more restrictive as the query advances, instead of sorting all elements and picking the highest k.

Queries

`BI 2` `BI 4` `BI 5` `BI 9` `BI 16` `BI 19` `BI 22` `IC 11`

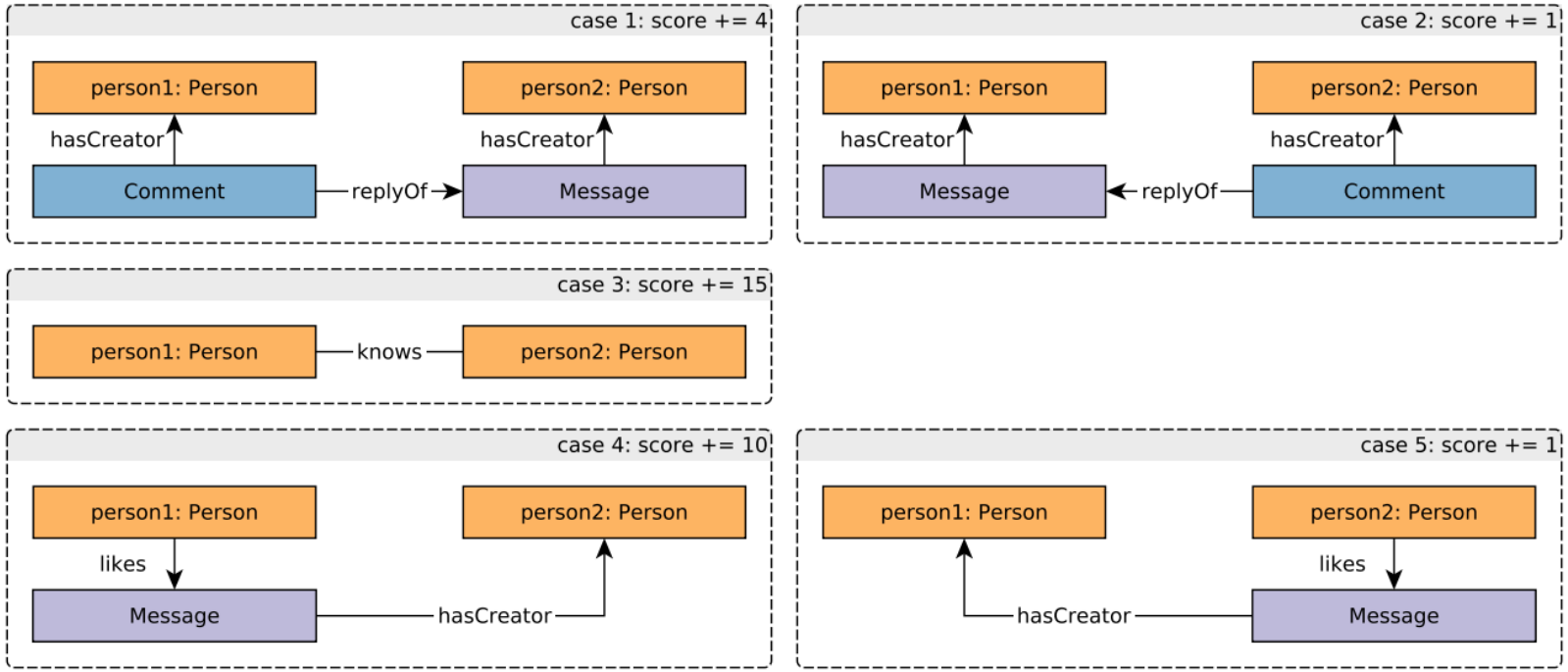
Q22: International dialog



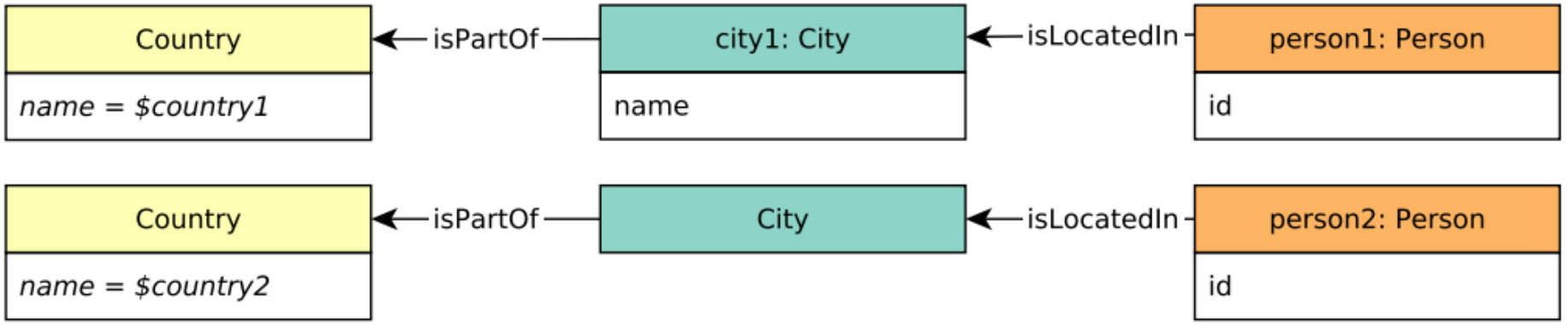
For each p1-p2 pair, calculate score and get top pair (w/ tie-break)

- `+ 4` if p1 replied to p2
- `+ 1` if p2 replied to p1
- `+15` if p1 knows p2
- `+10` if p1 liked p2's msg
- `+ 1` if p2 liked p1's msg

= max. 31 in total



Q22: International dialog




```
+ 4  if p1 replied
+ 1  if p2 replied
+15  if p1 knows p2
+10  if p1 liked
+ 1  if p2 liked
```

= max. 31 in total

Avoiding full Cartesian product with Top-*k* pushdown:

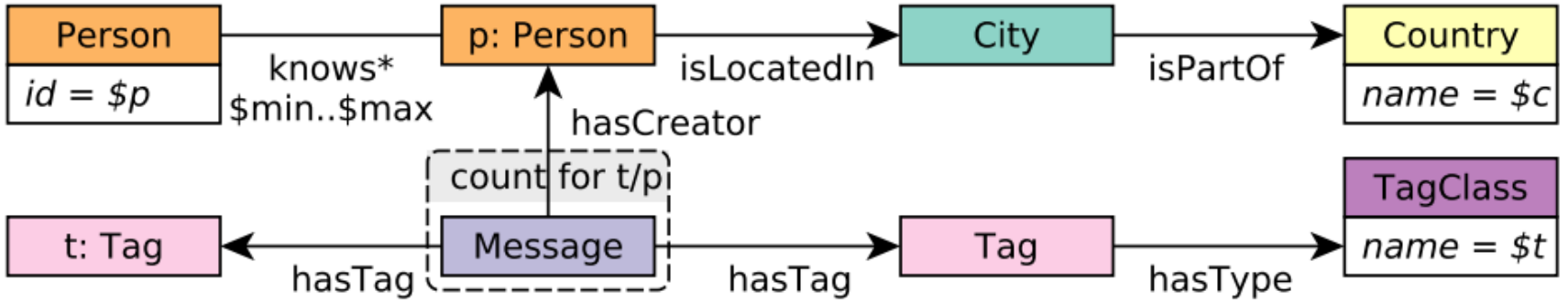
Example #1:

- There are *k* pairs with maximum points (31).
- A pair cannot possibly achieve max. points → prune

Example #2:

- There are *k* pairs with at least 20 points.
- A pair fails the condition for 15 points → prune

Q16: Experts in social circle



1	messageCount	↓
2	tag.name	↑
3	person.id	↑

- CP-1.3 [QOPT] Top-*k* pushdown
- CP-7.1 [QEXE] Path pattern reuse
- CP-7.2 [QOPT] Cardinality estimation of transitive paths
- CP-7.3 [QEXE] Execution of a transitive step

Baseline	29 s
Top-k	27 s
Top-k + Path pattern reuse	15 s

Sparksee run times on SF10

Language choke points

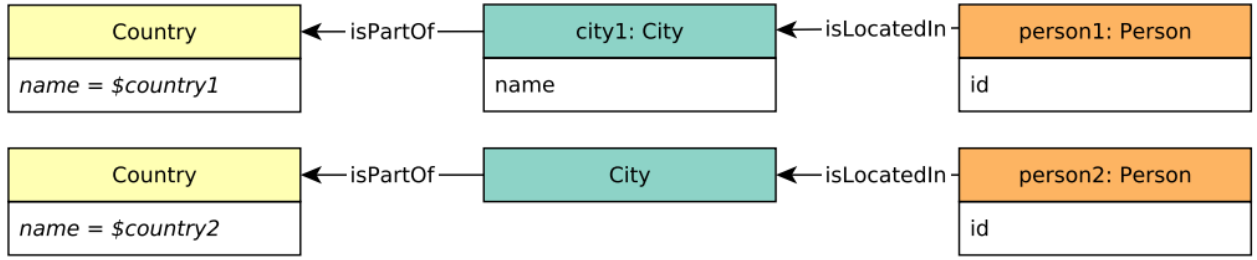
New choke points to cover *language features*.

- CP-8.1: Complex patterns
- CP-8.2: Complex aggregations
- CP-8.3: Ranking-style queries
- CP-8.4: Query composition
- CP-8.5: Dates and times
- CP-8.6: Handling paths

Language choke points

New choke points to cover *language features*.

- CP-8.1: Complex patterns
- CP-8.2: Complex aggregations
- CP-8.3: Ranking-style queries
- CP-8.4: Query composition
- CP-8.5: Dates and times
- CP-8.6: Handling paths



Q22: select top pair for each `city1`

"`LIMIT 1`" not sufficient

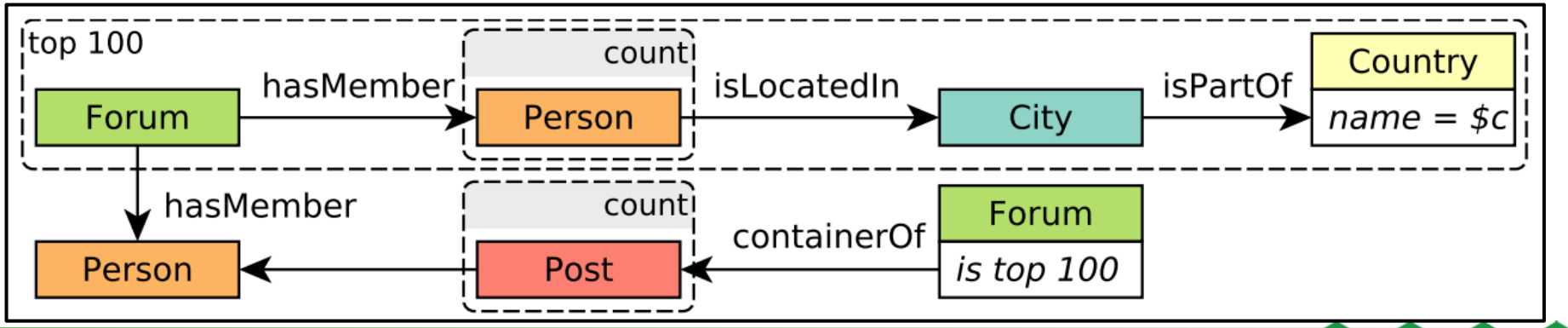
PostgreSQL: `rank()`

Language choke points

New choke points to cover *language features*.

- CP-8.1: Complex patterns
- CP-8.2: Complex aggregations
- CP-8.3: Ranking-style queries
- CP-8.4: Query composition
- CP-8.5: Dates and times
- CP-8.6: Handling

Q5: top 100 forums
(Important feature of G-CORE.)



Language choke points

New choke points to cover *language features*.

- CP-8.1: Complex patterns
- CP-8.2: Complex aggregations
- CP-8.3: Ranking-style queries
- CP-8.4: Query composition
- CP-8.5: Dates and times
- CP-8.6: Handling paths

Q1: aggregate for each month

"Datetime" features:

- SQL ✓
- SPARQL ✓
- Cypher: recently added ✓

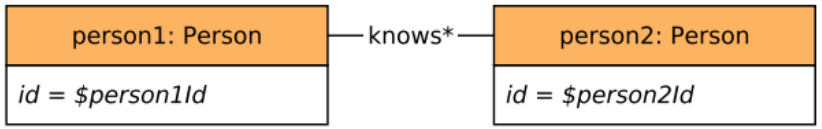
Language choke points

New choke points to cover *language features*.

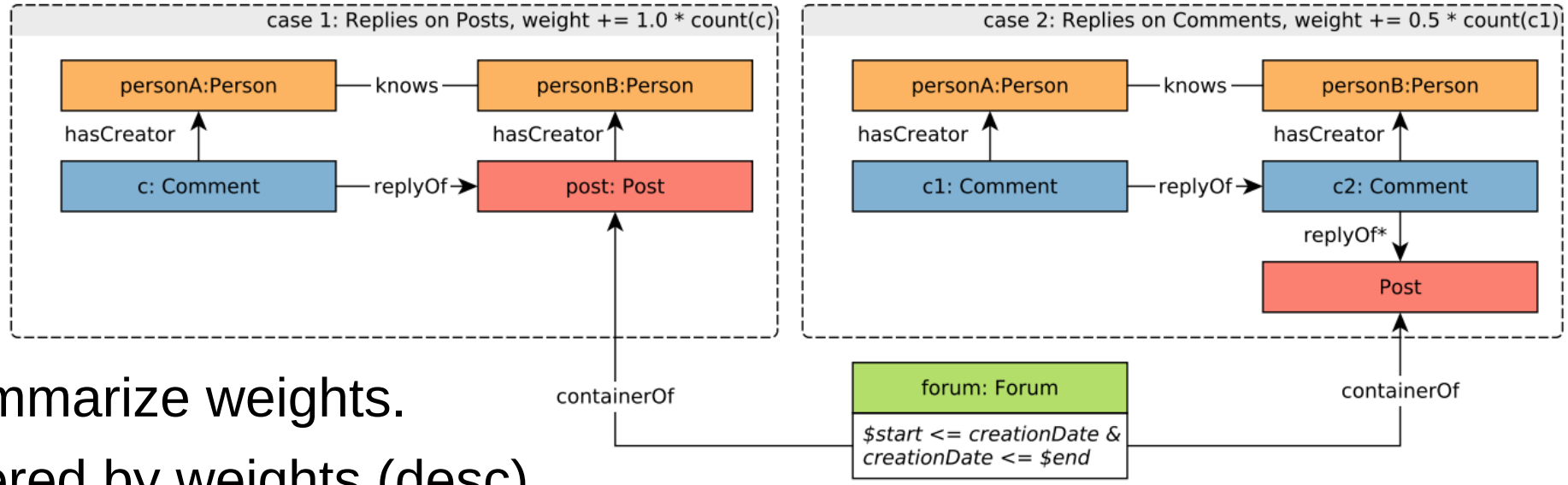
- CP-8.1: Complex patterns
- CP-8.2: Complex aggregations
- CP-8.3: Ranking-style queries
- CP-8.4: Query composition
- CP-8.5: Dates and times
- CP-8.6: Handling paths

Q25: Weighted interaction paths

1. Given two *Persons*, get all shortest paths on “knows” edges.



2. For each path, for each edge on the path, calculate a weight.



3. For each path, summarize weights.
4. Return paths, ordered by weights (desc).

(Q25 covers 15 CPs, incl. *all language-related ones* – its SQL impl. is ~2500 chars)

CP-8.6 Handling paths

1. Path unwinding: “higher-order” queries

   Q25: weights based on additional pattern matching on path elements.

2. Matching semantics for paths
   - Homomorphism-based (walks)
   - Isomorphism-based
     - No-repeated-anything
     - No-repeated-edge semantics (trails)
     - No-repeated-node semantics (simple path)

   Q16: “social circle” – persons connected by an edge-unique paths of [*x*, *y*] hops

3. Regular path queries (RPQs)

R. Angles et al.,
Foundations of Modern Query Languages for Graph Databases,
ACM Computing Surveys, 2017

CP-8.6 Handling paths

LDBC

CP-8.6: [LANG] Handling paths

Handling paths as first-class citizens is one of the key distinguishing features of graph database systems [3]. Hence, additionally to reachability-style checks, a language should be able to perform *path unwinding* [1], i.e. express queries that operate on elements of a path such as calculating a score for each edge of a path. Also, some use cases specify uniqueness constraints on paths, e.g. that a certain path must not have repeated nodes (referred to as "walks" in graph theory) or not have repeated edges ("trails" in graph theory). Following the definitions of paper [1], *homomorphism-based semantics* (no constraints on repetitions) and multiple flavours of *isomorphism-based semantics* (no-repeated-node, no-repeated-edge, and no-repeated-anything).

Cypher. Cypher uses *no-repeated-edge matching semantics* (in return, this semantics is sometimes dubbed as *cyphermorphism*). Configurable matching semantics (e.g. `MATCH ALL WALKS`) were proposed in the openCypher language. RPQs are also proposed in the openCypher language as *path patterns*.

G-CORE. G-CORE treats paths as *first-order citizens*: its *path property graph data model* can store paths in the graph model itself. However, the language only supports shortest path semantics (for tractability reasons) and does not allow enumeration of all paths. G-CORE uses *homomorphism-based matching semantics*.

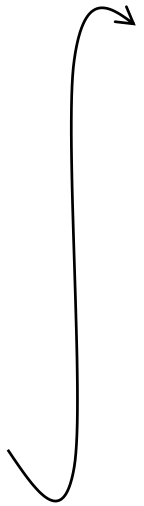
SPARQL. SPARQL uses *homomorphism-based matching semantics* and supports RPQs as *property paths*. Isomorphism-based matching semantics can be expressed by introducing custom filtering condition on predicates, e.g. `FILTER ( ?e1 != ?e2 ).`

Implementing the BI workload

LDBC

High-level process

1. Generate data set
2. Implement loader
3. Implement driver adapter
4. Implement queries and validate



Validation

1. Generate validation set
2. Cross-validate for multiple SFs
3. Failure → fix issues and go to 2

Very time consuming process, but…

- after 2 validated tools – still bugs in *both* implementations
- after 3 validated tools – still ambiguities in the spec

Implementing the BI workload

Cross-validation for implementations

Cypher	Neo4j	25/25
SQL	PostgreSQL	25/25
Imperative	Sparksee	25/25
SPARQL	Stardog	24/25
PGQL	Oracle Labs PGX	10/25

In progress: Spark SQL

Roadmap

1. **Help industry adoption** → get more benchmark results

2. **Define updates on the graph.**
   Necessitates complex dependency handling (SIGMOD'15 paper), and raises many design choices:
   - Affected types — which nodes/edges? what distribution?
   - Nature of changes — append-only vs. insert and delete
   - Granularity — nodes/edges vs. attributes
   - Frequency of changes — streaming vs. batch

3. **Publish as a conference paper**

Acknowledgements

Gábor Szárnyas was partially supported by NSERC RGPIN-04573-16 (Canada) and the MTA-BME Lendület Cyber-Physical Systems Research Group (Hungary).

DAMA-UPC research was supported by the grant TIN2017-89244-R from MINECO (Ministerio de Economia, Industria y Competitividad) and the recognition 2017SGR-856 (MACDA) from AGAUR (Generalitat de Catalunya).

Sparsity thanks the EU H2020 for funding the Uniserver project (ICT-04-2015-688540).

Hungarian Academy of Sciences

MTA-BME Lendület
Cyber-Physical Systems Research Group

Department of Measurement and Information Systems

McGill

Department of Electrical and Computer Engineering